



Member-only story

A Production-Grade GraphRAG Architecture with Neo4j and LangChain



Akash Goyal

Follow

13 min read · Dec 13, 2025



99



The Convergence of AI and Knowledge Graphs



Image by Author | generated using Gemini

In this blog, you will learn how to take a raw Neo4j database and turn it into a fully functional, production-ready GenAI application platform using the LangChain Neo4j package. You will see exactly how to set up a containerized Neo4j instance with automatic database initialization, configure vector indexes for semantic search with 1536-dimensional embeddings, and build a complete retrieval-

augmented generation pipeline. The article also shows how to expose a natural language interface that converts plain English questions into precise Cypher queries and how to persist entire conversation histories as interconnected graph structures for long-term memory and analytics. By the end, you will have a clear, repeatable blueprint for deploying high-performance graph-powered AI applications with full control over data structure, retrieval accuracy, and conversational context.

The Infrastructure Foundation

The foundation of this platform is a production-ready Neo4j 5.26 Enterprise database running in Docker. The infrastructure is configured through a docker-compose file that handles automatic database initialization, authentication setup, and plugin installation without manual intervention. The system uses marker files to track initialization state, ensuring that operations like database loading and authentication configuration only run once, even across container restarts. The setup leverages the APOC plugin for advanced graph operations and initializes with a pre-populated recommendations database containing movies, actors, directors, and pre-computed vector embeddings.

The database initialization script performs several critical operations in sequence. First, it checks for an authentication marker file and if not found, it resets the auth store and configures the initial password from the NEO4J_AUTH environment variable. Then it looks for a database loading marker and if absent, it uses neo4j-admin to load

the recommendations database from a backup dump file. After starting the Neo4j server process in the background, it waits for the server to become available and then executes a Cypher command to set the database access mode to read-write. This automated workflow ensures that developers can start working with a fully configured system immediately after running docker-compose up.

The recommendations database comes with a rich schema that models the relationships between movies and the people who create them. The schema includes Movie nodes with properties like title, plot, and released date, along with Actor and Director nodes containing biographical information. These nodes are connected through ACTED_IN and DIRECTED relationships that capture the role of each person in each film. What makes this database particularly powerful for AI applications is the embedding property on each Movie node, which contains a 1536-dimensional vector representation of the movie plot generated using OpenAI's text-embedding-ada-002 model. These pre-computed embeddings enable instant semantic search without requiring runtime embedding generation.

The Knowledge Graph Layer

At the core of the system lies the Knowledge Graph layer, managed through the Neo4jGraph component. This component acts as the primary interface for all graph operations, handling connection management, query execution, and schema introspection. The connection configuration requires specifying the Bolt protocol

endpoint at `bolt://localhost:7687`, authentication credentials, and the target database name. Unlike traditional database abstractions that hide the underlying data model, Neo4jGraph provides full access to the graph schema through its `get_schema` method, which returns detailed information about all node labels, relationship types, and their associated properties.

The flow begins with the data ingestion process, which takes structured information and models it as interconnected nodes and relationships. Instead of writing raw Cypher queries, developers define their graph structure using Python objects from the `langchain_neo4j.graphs.graph_document` module. The `Node` class represents entities with a type label and a properties dictionary, while the `Relationship` class captures connections between nodes with its own type and properties. These objects are wrapped in a `GraphDocument` that ensures atomicity when committing to the database. This object-oriented approach provides type safety, better error messages, and cleaner code compared to string-based query construction.

Image by Author | generated using Gemini

When you add new data to the graph, the system validates the structure and executes the insertion as a single transaction. For example, adding information about Timothée Chalamet and the movie Dune involves creating two Node objects with their respective properties, then creating a Relationship object that connects them with the ACTED_IN relationship type and includes the specific role as

a property. The `graph.add_graph_documents` method accepts a list of `GraphDocument` objects, allowing you to insert multiple disconnected or connected subgraphs in a single operation. This batch capability is essential for efficient data loading when building knowledge bases from external sources like Wikipedia, corporate databases, or content management systems.

Image by Author | generated using Gemini

```
from langchain_neo4j import Neo4jGraph
from langchain_neo4j.graphs.graph_document import GraphDocument, Node, F
# Initialize the graph connection
graph = Neo4jGraph(
    url="bolt://localhost:7687",
    username="neo4j",
    password="password",
    database="recommendations",
)
# Define nodes with properties
timotheeNode = Node(
    id=1,
    type="Actor",
    properties={
        "name": "Timothée Chalamet",
```

```

        "born": "1995-12-27",
    },
)
duneNode = Node(
    id=2,
    type="Movie",
    properties={
        "title": "Dune",
        "released": "2021-09-03",
    },
)
# Define the relationship with role information
actedInRelationship = Relationship(
    source=timotheeNode,
    target=duneNode,
    type="ACTED_IN",
    properties={
        "role": "Paul Atreides",
    },
)
# Commit to database
graph.add_graph_documents(
    graph_documents=[
        GraphDocument(
            nodes=[timotheeNode, duneNode],
            relationships=[actedInRelationship]
        )
    ]
)
# Query to verify
result = graph.query(
    "MATCH (a:Actor {name: 'Timothée Chalamet'})-[r:ACTED_IN]->(m:Movie)"
)
print(result)

```

Output:

```

[
  {
    'a': {'name': 'Timothée Chalamet', 'born': '1995-12-27'},
    'r': {'role': 'Paul Atreides'},

```

```
    'm': {'title': 'Dune', 'released': '2021-09-03'}  
  }  
]
```

The query result demonstrates how Neo4j returns the complete node and relationship properties as nested dictionaries. Each element in the result list represents one match from the graph, with keys corresponding to the variables defined in the RETURN clause of the Cypher query. This structured output makes it easy to extract specific properties or traverse the relationship chain programmatically.

The Retrieval Layer

The retrieval layer is powered by Neo4jVector and is responsible for serving relevant context to the language model with high precision. This component integrates the semantic understanding of vector embeddings with the structural richness of the knowledge graph through a sophisticated dual-mode architecture. The system first creates a vector index on the Movie nodes using the CREATE VECTOR INDEX command, specifying the embedding property as the source of the 1536-dimensional vectors and configuring the cosine similarity function for computing distances between embeddings.

When configuring the Neo4jVector store, you must specify several parameters that map the vector search to your graph schema. The embedding parameter takes an OpenAIEmbeddings instance configured with the text-embedding-ada-002 model, ensuring that

query embeddings match the dimensionality and semantic space of the stored vectors. The `index_name` parameter must match the name used when creating the vector index in the database. The `node_label` specifies which type of nodes contain the vectors, while `embedding_node_property` identifies which property holds the vector data. The `text_node_property` tells the system which property contains the original text that was embedded, allowing it to return human-readable content in search results.

```
from langchain_neo4j import Neo4jVector
from langchain_openai import OpenAIEmbeddings
# Configure the embedding model
embedding = OpenAIEmbeddings(model="text-embedding-ada-002")
# Connect to existing vector index
vector_store = Neo4jVector.from_existing_index(
    embedding=embedding,
    index_name="moviePlots",
    node_label="Movie",
    embedding_node_property="embedding",
    text_node_property="plot",
    graph=graph,
)
# Perform semantic search
results = vector_store.similarity_search(
    "Movie about a sentient cowboy doll",
    k=3
)
for doc in results:
    print(f>Title: {doc.metadata['title']}")
    print(f>Plot: {doc.page_content[:150]}...")
    print("----")
```

Output:

```
Title: Toy Story
Plot: A cowboy doll is profoundly threatened and jealous when a new spac
----
Title: Toy Story 2
Plot: When Woody is stolen by a toy collector, Buzz and his friends vow
----
Title: Toy Story 3
Plot: The toys are mistakenly delivered to a day-care center instead of
----
```

The semantic search accurately identifies all three Toy Story movies despite the query not mentioning “Toy Story” explicitly. The embedding model understands that “sentient cowboy doll” semantically relates to the concept of living toys and Western characters, demonstrating the power of vector search for capturing intent beyond keyword matching. Each result includes the original plot text and metadata from the Movie node, providing complete context for the language model.

The system also supports creating new vector indexes from scratch using the `from_documents` method. This workflow is particularly useful when adding new types of content to your knowledge graph. The method accepts a list of Document objects, each containing `page_content` that will be embedded and stored. Behind the scenes, the system creates the appropriate node label, generates embeddings for all documents using the specified embedding model, creates the vector index with optimal configuration, and stores both the original text and the computed embeddings as node properties. This automated workflow reduces the complexity of setting up vector

search and ensures consistency across all indexed content.

```
from langchain_core.documents import Document
# Create documents about AI frameworks
docs = [
    Document(
        page_content="LangChain is a framework to build with LLMs by chaining interoperable components",
    ),
    Document(
        page_content="LangGraph is an open-source framework for building stateful, multi-agent applications",
    ),
]
# Create new vector index
new_vector_store = Neo4jVector.from_documents(
    docs,
    embedding,
    url="bolt://localhost:7687",
    username="neo4j",
    password="password",
    database="recommendations",
    index_name="libraries",
    node_label="Library",
)
# Query the new index
library_results = new_vector_store.similarity_search("What is LangChain?")
print(library_results[0].page_content)
```

Output:

```
LangChain is a framework to build with LLMs by chaining interoperable components
```

The vector search correctly retrieves the LangChain document when asked “What is LangChain?” even though the query is phrased as a

question and the document is a statement. This demonstrates how embeddings capture semantic meaning beyond surface-level text matching, making them ideal for question-answering systems where queries rarely match document text exactly.

Unlike simple vector stores that only return similar documents, the Neo4j integration allows you to immediately traverse graph relationships from the retrieved nodes. After finding relevant movies through semantic search, you can execute Cypher queries to discover connected actors, directors, genres, and user ratings. This hybrid retrieval approach ensures that the language model receives complete context grounded in both semantic similarity and factual relationships. The ability to combine vector operations with graph traversal in a single database eliminates the complexity of synchronizing data across multiple systems.

The Application Layer

Image by Author | generated using Gemini

The application logic is orchestrated by the GraphCypherQAChain, which transforms natural language into executable database queries through a sophisticated multi-step process. The chain uses a large language model to interpret the user's intent, examines the database schema to understand available node types and relationships, generates syntactically correct Cypher queries that answer the

question, executes the query against Neo4j, and finally synthesizes a natural language response from the structured results. This architecture effectively decouples the user interface from the underlying data structure while ensuring that answers are always derived from actual data in the graph rather than the model's training data.

The chain configuration requires several important parameters that control its behavior. The `allow_dangerous_requests` flag must be set to `True` as an explicit acknowledgment that LLM-generated queries could potentially be destructive or resource-intensive. The `return_intermediate_steps` parameter, when enabled, causes the chain to include the generated Cypher query in its output, which is invaluable for debugging, understanding how questions are interpreted, and validating query logic. The `return_direct` option allows you to skip the final language model call and return the raw query results, which is useful when building applications that need to process the data programmatically rather than displaying it to end users.

python

```
from langchain_neo4j import GraphCypherQAChain
from langchain_openai import ChatOpenAI
# Initialize the LLM
llm = ChatOpenAI(temperature=0.0, model="gpt-4o-mini")
# Create the chain with visibility into query generation
chain = GraphCypherQAChain.from_llm(
    llm=llm,
    graph=graph,
```

```

        allow_dangerous_requests=True,
        return_intermediate_steps=True
    )
    # Ask questions in plain English

    result = chain.invoke({"query": "When was the movie 'Heat' released?"})
    print("Generated Cypher Query:")
    print(result["intermediate_steps"][0]["query"])
    print("\nAnswer:")
    print(result["result"])

```

Output:

```

Generated Cypher Query:
MATCH (m:Movie {title: 'Heat'})
RETURN m.released

Answer:
The movie 'Heat' was released on December 15, 1995.

```

The output reveals the two-step process of the GraphCypherQChain. First, the LLM analyzes the question and generates a precise Cypher query that matches a Movie node by title and returns the released property. Then, it takes the raw query result and synthesizes a natural language answer that directly addresses the user's question. This transparency is crucial for debugging and building trust in the system's responses.

For production deployments requiring optimal cost and performance, the chain supports using different language models for different steps. You might use a more powerful model like GPT-4 for

generating accurate Cypher queries where precision is critical, while using a faster, cheaper model like GPT-3.5 for the final answer generation where the task is simpler. This separation of concerns allows you to optimize each step independently based on its specific requirements. The `cypher_llm` parameter controls query generation while the `llm` parameter controls answer synthesis.

```
# Use different models for different steps
cypher_llm = ChatOpenAI(temperature=0.0, model="gpt-4")
qa_llm = ChatOpenAI(temperature=0.0, model="gpt-3.5-turbo")
chain = GraphCypherQAChain.from_llm(
    llm=qa_llm,
    cypher_llm=cypher_llm,
    graph=graph,
    allow_dangerous_requests=True,
    return_intermediate_steps=True,
)
result = chain.invoke({"query": "Who directed the movie 'Heat'?"})
print(result["result"])
```

Output:

```
Michael Mann directed the movie 'Heat'.
```

The dual-LLM configuration allows you to leverage GPT-4's superior reasoning for query construction while using GPT-3.5's faster response times for the simpler task of formatting the answer. This architecture can reduce costs by up to 90 percent compared to using

GPT-4 for both steps, while maintaining query accuracy since the critical Cypher generation still uses the more capable model.

The Persistence Layer

The persistence layer is handled by Neo4jChatMessageHistory, which stores entire conversation flows directly within the graph database as linked node structures. Every message exchanged between the user and the AI is saved as a Message node with properties indicating the message type (human or AI), content, and timestamp. These message nodes are connected through NEXT relationships that form a chronological chain, with the most recent message linked to a Session node via a LAST_MESSAGE relationship. This graph-based storage pattern allows for efficient retrieval of conversation history while enabling sophisticated analysis through Cypher queries.

The conversation graph structure provides several operational advantages over traditional conversation storage. You can quickly retrieve all messages in a session by following the NEXT relationships from the last message backward through the chain. You can analyze conversation patterns across multiple sessions to identify common question types, track topic transitions, or measure response quality. You can create relationships between conversation nodes and domain entities mentioned in the discussion, enabling queries like “show me all conversations where users asked about this specific movie” or “find sessions where users discussed action films released after 2010.”

Image by Author | generated using Gemini

```
from langchain_neo4j import Neo4jChatMessageHistory
# Initialize conversation history
history = Neo4jChatMessageHistory(
    graph=graph,
    session_id="session_id_1",
)
# Add messages to the conversation
history.add_user_message("hi!")
history.add_ai_message("whats up?")
# Retrieve the conversation
messages = history.messages
for msg in messages:
    print(f"{msg.type}: {msg.content}")
```

Output:

```
human: hi!
ai: whats up?
```

The message retrieval demonstrates the sequential storage and retrieval of conversation turns. Each message preserves both its content and type, allowing the system to distinguish between user inputs and AI responses. This structured format is essential for feeding conversation context back to the language model in subsequent turns.

The real power of graph-based conversation storage emerges when you integrate it with the GraphCypherQAChain to create stateful conversational AI systems. The RunnableWithMessageHistory wrapper connects any LangChain chain with the conversation history manager, automatically loading context at the start of each interaction and saving new exchanges after completion. This integration requires specifying which input and output keys correspond to messages, allowing the wrapper to correctly extract and store the conversation flow.

```
from langchain_core.runnables.history import RunnableWithMessageHistory
# Create the base chain
llm = ChatOpenAI(temperature=0.0, model="gpt-4o-mini")
chain = GraphCypherQAChain.from_llm(
    llm=llm,
    graph=graph,
    allow_dangerous_requests=True
)
# Wrap with conversation history
chain_with_history = RunnableWithMessageHistory(
    chain,
    lambda session_id: Neo4jChatMessageHistory(graph=graph, session_id=session_id,
    input_messages_key="query",
    output_messages_key="result",
)
# Use with session tracking
```

```

response = chain_with_history.invoke(
    {"query": "When was the movie 'Heat' released?"},
    config={"configurable": {"session_id": "session_id_2"}},
)
print("First query:", response["result"])
# Subsequent queries in the same session maintain context
followup = chain_with_history.invoke(
    {"query": "Who was the director?"},
    config={"configurable": {"session_id": "session_id_2"}},
)
print("Follow-up query:", followup["result"])

```

Output:

```

First query: The movie 'Heat' was released on December 15, 1995.
Follow-up query: Michael Mann was the director of 'Heat'.

```

The follow-up query demonstrates the power of conversation memory. When asked “Who was the director?” without specifying which movie, the system understands from the conversation context that the user is still asking about Heat. The `RunnableWithMessageHistory` automatically provides the previous messages to the chain, allowing the language model to resolve the pronoun reference and generate the correct Cypher query for the intended movie.

The conversation persistence system supports advanced cleanup operations through its `clear` method. When you call `clear` with `delete_session_node` set to `False`, it removes all message nodes but

preserves the session node, which is useful when you want to reset a conversation while maintaining session metadata. Setting `delete_session_node` to `True` removes both the messages and the session node, completely erasing all traces of that conversation from the graph. This granular control over data lifecycle is essential for applications that must comply with data retention policies or privacy regulations.

Why This Setup Works

This setup is chosen because it balances flexibility, accuracy, and detailed context in a clean and production-ready way. Neo4j ensures that your data relationships are first-class citizens with efficient traversal and rich querying capabilities. LangChain delivers the orchestration logic to connect language models with your data through well-tested abstractions. The hybrid retrieval approach provides the best of both vector search and graph traversal, enabling systems that understand both semantic similarity and factual connections. The conversation graph pattern turns user interactions into queryable data structures that can drive analytics and improve system behavior over time.

Together, these components form a robust architecture that is easy to extend, easy to reason about, and safe to deploy for mission-critical applications where accuracy and context are paramount. The containerized infrastructure ensures consistent deployment across development and production environments. The vector search capabilities enable semantic understanding without sacrificing the

precision of structured queries. The natural language interface makes complex graph data accessible to non-technical users. The conversation persistence transforms ephemeral interactions into permanent knowledge that can inform future decisions and uncover usage patterns.

Langchain

Neo4j

Knowledge Graph

Generative Ai Tools

Artificial Intelligence



Written by Akash Goyal

Follow

889 followers · 216 following

Building Agentic AI applications | Generative AI | LLMs | Agents | NLP | Data Scientist | Machine Learning <https://www.linkedin.com/in/akash-g-7a5224246/>

More from Akash Goyal

 Akash Goyal

Building Agentic GraphOS: The 16-Layer Architecture Behind...

A Cost-Optimized, Fully Observable GraphRAG Operating System Built for...

★ Dec 14, 2025 🖱 593 💬 6 📌 ⋮

 Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval...

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform...

★ Dec 13, 2025 🖱 265 💬 4 📌 ⋮

 Akash Goyal

Why LLMs Keep Forgetting Ontologies (And How to Fix It)

Production Architecture for Ontology Memory (Neo4j + LangGraph)

★ Dec 23, 2025 🖱 238 💬 4 📌 ⋮

 Akash Goyal

A Complete Guide to Agentic Context Graphs: Turning...

Why capturing the “why” behind decisions is the next great leap in...

★ Jan 10 🖱 147 💬 2 📌 ⋮

See all from Akash Goyal

Recommended from Medium



In Towards AI by Gaurav Shrivastav

Building RAG Systems in 2026 With These 11 Strategies

Why your RAG is failing and how
combining these techniques can fix it



Dec 12, 2025



434



7



Vishal Mysore

Agentic Knowledge Graphs with A2UI: Why AI Reasoning...

Agentic Knowledge Graphs represent a
new class of AI-driven visualization...

5d ago



66



1



 Fanghua (Joshua) Yu

A Unified Framework for AI-Native Knowledge Graphs

Part 1—Generative Knowledge Modeling

4d ago  22  1



 Pankaj Kumar

Building Your First Ontology: A Hands-On Tutorial

Stop Reading About Ontology. Start Building One.

 Dec 13, 2025  527  6  

 Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval...

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform...

 Dec 13, 2025  265  4  

 In AI Advances by Lore Van Oudenhove

Building AI Agents with LangGraph (2026 Edition): A...

Over the past two years, LangGraph has become the core of almost everything I...

Jan 3  462  3  

See more recommendations

